

Extraction of Step Performed in Use Case Description as a Reference for Conformity of Sequence Diagrams Using Text Mining (Case Study: SRS APTU)

Nur Apriyanto

Department of Informatics
Telkom University
Bandung, Indonesia

nurapriyanto@student.telkomuniversit
y.ac.id

Yudi Priyadi

Department of Software Engineering
Telkom University
Bandung, Indonesia

whyphi@telkomuniversity.ac.id

Dana Sulistyo Kusumo

Department of Software Engineering
Telkom University
Bandung, Indonesia

danakusumo@telkomuniversity.ac.id

Abstract— Extraction is an essential part of processing a document to ensure the success of the text mining process. In this study, the example of the SRS document used is the Integrated Service Application (APTU) KPKNL Bandung, an application to manage the process of submitting service tickets at the State Property and Auction Service Office. There is a difference in interpreting the activities that exist in the Use Case Description artifact with a Sequence Diagram that provides an overview of the functionality of a process to show the involvement of an activity related to the Use Case Description. This study aims to perform step extraction on the Use Case description. The results of this extraction are compared for their suitability with the sequence diagram using the concept of text mining. There are core results from this research activity. First, the highest similarity between documents is in the SP01 and SD01 documents, with the similarity value being 0.69108792. Second, the highest similarity between words is found in words "list" and "menu," with the similarity value being 0.9412. Third, the Kappa Score from Gwet's AC1 formula using the Python programming language is 0.12362, which means "Slight Agreement," while the Kappa Score value using a questionnaire filled in by the expert is 0.97464, which means "Almost perfect."

Keywords— *Extraction, Step Performed, Use Case Description, Sequence Diagram, Text Mining, Similarity.*

I. INTRODUCTION

Extraction is a process of finding the feature values in documents that are useful in the text mining process. Extraction is a crucial part of processing a document to ensure the success of the text mining process [1]. Meanwhile, text mining is a process of exploring and analyzing to get information from a set of data in the form of text and unstructured data, or at least semi-structured. The exploration process of text mining continued in doing text similarity [2][19].

This similarity process makes a match between the steps performed and the sequence diagram. Cosine similarity is one of the measurement metrics used to measure the similarity between sentences related to each other and determine a score based on similar words in the sentence.

Software requirements specification (SRS) is a document that explains how to develop software. In simple terms, SRS contains documentation about the details of the application to be worked on. It not only describes the software that will be

made, but it must contain information about the purpose of manufacture [3][4][5]. In the SRS document, there are functional requirements specifications, namely to describe the functionality of the system or system services. In addition, there are non-functional requirements used to determine the attributes or overall quality of a system. The SRS also includes use case diagrams, use case descriptions, and sequence diagrams.

In this study, the example of the SRS document used is the Integrated Service Application (APTU) KPKNL Bandung, an application to manage the process of submitting service tickets at the State Property and Auction Service Office. Table I, examples of interpreting the requirements in the SRS document for the APTU Application. There is a difference in interpreting the activities in the Use Case Description artifact with a Sequence Diagram that provides an overview of the functionality of a process to show the involvement of an activity related to the Use Case Description.

TABLE I EXAMPLE OF INTERPRETATION BETWEEN STEP PERFORMED AND SEQUENCE DIAGRAM

Step Performed	Sequence Diagram
1. Admin selects the "Manage Section" menu	1. Select Menu Section, display section, fill in form add section, input message is invalid, display list section, select menu section, display page section, input is invalid, Invalid input message, display section list, display section list, fill form add section, data section, display list section, display list section, data section, input no valid, Save Section data, display Section list, display Section list, display Section list.
2. The system displays the view section	
3. Admin selects the "Add" button	2. Etc.
4. Etc.	

Therefore, to overcome this problem, a process will be conducted to find similarities from the APTU Software requirements specification document, use data processing, and check the validity and reliability of Search Results. This study uses a similar calculation based on context or meanings between words using Word similarity for Java application tools.

In this study, the purpose was to extract the steps performed on the Use Case description, then compare the

results of this extraction with the sequence diagram using the concept of text mining.

In this study, there are contributions and novelties, as follows:

- Extract the steps performed on the use case description and sequence diagram.
 - Implementing text preprocessing on the extracted step performed and sequence diagrams.
 - Based on the extraction results, the similarity process can be carried out between the steps performed in the use case description and the sequence diagram.
- Validate through Agreement Coefision (Gwet's AC1).

II. RELATED WORKS

A. Software Requirement Spesification (SRS) Document

SRS document is an abbreviation of software requirements specification, which is a document that contains an explanation of how to develop software. In simple terms, SRS includes a roadmap of all parties involved in a software development project [1][17][18].

The SRS document describes the abbreviation of software requirements specification as a document that contains a description of the software development method. In simple terms, SRS includes a roadmap of all parties participating in a software development project [3].

Use case diagram is a diagram that describes the relationship between the actor and the system. Use case diagrams can describe an interaction between one or more actors with the system to be created. The benefits of use case diagrams are to describe the application's system requirements. While the understanding of the use case description is one of the use case diagrams that aims to provide an overview of the general process of the functionality of the use case diagram [4].

Sequence Diagrams describe the interactions between objects and around the system in the form of messages that are depicted against current events. Sequence diagrams illustrate scenarios or flows that are carried out in response to events to produce a specific output. The sequence diagram begins with selecting the activity that will cause the movement, carrying out the appropriate process, and what changes occur in input or output. Each object, including the actor, has a lifeline in the form of a vertical line. The message is described as a line with arrows from actor to object or object to object or vice versa[1][4][5].

B. Cosine Similarity

Cosine similarity is one of the measurement metrics used to measure the similarity between sentences related to each other and determine a score based on similar words in the sentence. The angle between two vectors determines the cosine similarity, allowing document data with the same composition to be processed identically[6][7][8][16]. Through the calculation process, it is possible to find similarities between documents. The following is the calculation of the Cosine similarity value:

$$\text{similarity} = \cos(\theta) = \frac{A \cdot B}{\|A\| \|B\|} = \frac{\sum_{i=1}^n A_i B_i}{\sqrt{\sum_{i=1}^n A_i^2} \sqrt{\sum_{i=1}^n B_i^2}} \quad (1)$$

C. Gwet's AC1

Gwet's AC1 is a statistical calculation method to test the validity and reliability by measuring the results of the agreement index between two experts or experts in testing a recommendation model. The steps that must t to implement "the Gwet's" AC1 method [9][10][2] are as follows:

- Make a table of results elicitation of needs between two experts or experts by referring to the formula in Table II below.

TABLE II ELICITATION STATEMENT RESULTS

Expert 1	Expert 2		
	Yes	No	Yes
Yes	A	B	B1 = A+B
No	C	D	B2 = C+D
Total	A1 = A+C	A2 = B+D	N

- Calculate the observed agreement (P) using the following formula.

$$P = \frac{A + D}{N} \quad (2)$$

- Calculate the chance-agreement probability or $e(Y)$.

$$e(Y) = 2P_1(1 - P_1) \quad (3)$$

where for the formula one as follows:

$$P_1 = \frac{(A_1 + B_1)/2}{N} \quad (4)$$

Information:

A1 is the number of experts 1 and 2. At the same time, N is the number of data.

- Represent the results of the possible expert estimates by grouping the data into the "Yes" category.
- Calculates AC1 statistics with the coefficient agreement formula.

$$AC1 = \frac{P - e(Y)}{1 - e(Y)} \quad (5)$$

- Equating the results of AC1 statistics based on Table III regarding the interpretation of the value of Gwet's AC1.

TABLE II GWET'S AC1 VALUE INDEX

Kappa Index	Promotion Agreement
< 0.00	Less than chance-agreement.
0.01 – 0.20	Slight agreement.
0.21 – 0.40	Fair agreement.
0.41 – 0.60	Moderate agreement.
0.61 – 0.80	Substantial agreement.
0.81 – 1.00	Almost perfect.

D. Text Mining in Natural Languages and Text Processing

The algorithm that identifies and manages text is Natural Language Processing (NLP). NLP is a collection of syntactic processing algorithms applying semantic and statistical-based rules that can use to parse, segment, extract or analyze text data [11][18][23].

Text mining is a process of exploring and analyzing to obtain information from a set of data in the form of text, and this type of data is unstructured. In general, four processes need to carry out in the text mining process, including data collection, Text Preprocessing, designing a model that will carry out an iterative process through the evaluation and validation process, making presentations and interactions to verify the modeling results [12][20][21]. The text mining process starts from a collection of documents, and text mining

will take certain documents and then process them first by checking the writing format and character set. Then it will go through the text analysis stage, and sometimes the analysis technique needs to be repeated several times until the information is obtained [13][14][22].

III. METHODOLOGY

In this research activity, there is a gradual process to achieve the objectives and results of the discussion, namely: analysis of research objects, similarity searches, and validity testing. The stages of research activities can be seen in Figure 1.

A. Research Object Analysis

The analysis of the research object is divided into four processes, namely:

- Information gathering at this stage involves three activities, namely: interviews and discussions.
- Artifact modeling. The artifact modeling will be used in the step performed on the use case description and sequence diagram. In the first stage, the use case diagram is made. Then proceed to make a use case description that refers to the use case diagram. In the second stage, a sequence diagram is created.
- Extraction. After completing the modeling, proceed to the extraction process stage. The first step is to extract the use case description based on the step performed. The second step is to extract the sequence diagram.
- Text Pre-processing. This process is carried out using the Text Mining method to process and analyze the data that has been generated from the extraction process, which will be continued to find the similarity value. In this Text Pre-processing, there are six stages: defining the library, doing case folding, defining the dataset to be processed, tokenization, stemming, and stopwords[15].

B. Similarity search process

The similarity search process uses the cosine similarity formula to get the similarity value from the data set that has been processed in the pre-processing of the data.

C. Validity and Reliability Testing

The validity and reliability testing stage is the stage to test similar results that have been done previously. Validity and reliability testing was carried out using Gwet's AC1 formula. The AC1 formula is divided into three calculation stages: calculating the observed AC1, calculating the probability of agreement of the odds, and calculating the AC1 statistic. If the statistical value $AC1 > 0.8$ is declared valid, if the value is < 0.8 it is declared invalid [7]. Before testing the validity and reliability, data processing was carried out by distributing questionnaires to experts. These experts are people who have experience in the field of Software Engineering. The questionnaire given to the expert contains a similarity value between Step Performed in the Use Case Description and Sequence Diagram so that the results obtained from testing using Gwet's AC1 will have a comparison between the results of the Python program code and the results of the questionnaire filled out by the expert.

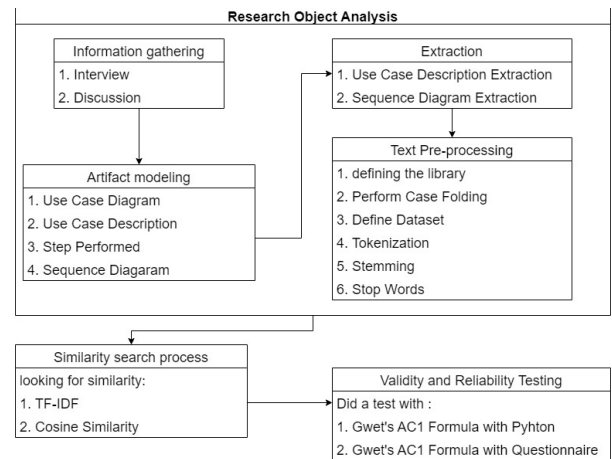


Fig. 1. Activity Stages

IV. RESULTS AND DISCUSSION

A. Artifact Modelling

In the APTU application, there is a use case diagram called "Service Ticketing Application at KPKNL Bandung." This SRS artifact contains 28 use cases, 29 use case descriptions, and 29 sequence diagrams. Not all artifacts can present in the discussion of this article. For an example of the discussion of data processing in this paper, only 10 data sets are used, consisting of 5 Steps Performed and five series of Sequence Diagrams.

Use Case Description and Sequence Diagrams on the SRS APTU application are the artifacts used. In Table IV, the following is an example of the presentation display for the Use Case Description of the APTU application.

TABLE IV USE CASE DESCRIPTION

User Name	Manage Section	Id: ADM-02
Actor	Admin	
Description	Manage sections by adding, changing, and deleting section data	
Step Performed	- Admin selects the "Manage Section" menu - The system displays the view section - Admin selects the "Add" button - The system displays an added section form - Admin inputs the form, it's more section - The system saves section data and displays it in the table - Admin chooses to change section item - The system displays section data on the edit form - Admin inputs data changes - The system saves data changes - Admin chooses to delete section items - System deletes section items	
Preconditions	Actor is logged in	
Postconditions	The actor has managed the section	
Assumptions	None	

In Figure 2, the following is an example of a Sequence Diagram presentation that describes a Manage Section feature in the admin section of the APTU application. This diagram consists of 5 lifelines: Admin, Menu Section, Add Section, Section Data, and ends at the Lifeline section of the Section Page.

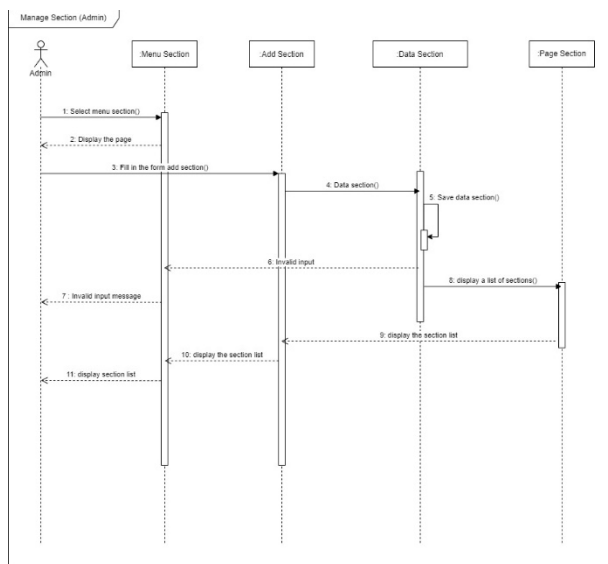


Fig. 2 Sequence Diagram

B. Extraction

The extraction process is carried out in the Use Case Description and Sequence Diagram. The extraction results from the use case description, based on the steps performed, are divided into several documents according to the existing features in the APTU application. Likewise for Sequence Diagrams. The extraction results were then coded according to their type. The following is an example of defining and explaining the code used by artifacts in SRS. See Table V.

TABLE III EXTRACTION DATA

No	Step Performed		Sequence Diagram	
	ID	Name of Use Case Diagram	ID	Name of Sequence Diagram
1	SP01	Admin manage section	SD01	Admin manage section
2	SP02	Supervisors view ratings and comments	SD02	Supervisors view ratings and comments
3	SP03	Ticket approval by APT Officer	SD03	Ticket approval by APT Officer
4	SP04	Service User submits a ticket	SD04	Service User submits a ticket
5	SP05	Service users view history	SD05	Service users view history

The following is an example of data extraction from Step Performed from the Use Case Description used, as follows:

- SP01 = "Admin selects the Manage Section menu, the system displays the section list, the admin selects the Add button, the system displays the added section form, the admin inputs the added section form, the system stores the section data and displays it in the table, the admin chooses to change the section item, the system displays the section data on the edit form, Admin inputs data changes, System saves data changes, Admin selects delete section items, System deletes section items."
- SP02 = "Supervisor selects the Rating Results menu, the System displays the rating results page, the Supervisor selects the Comments button, the system displays comments from the ticket rating."
- SP03 = "The APT officer selects the Approval Ticket menu, the system displays a list of entry tickets, the APT

officer selects the Approve button, the system processes and changes the ticket status."

- SP04 = "The Service User selects the Ticket menu, the system displays the ticket list page, the service user selects the Add Ticket button, the system displays the ticket form, the service user fills out the ticket form, the system processes and stores ticket data."
- SP05 = "The service user selects the Ticket History menu, the system displays a completed ticket list page, the service user selects a ticket, the system displays a rating page if the ticket has not been rated, the service user sends ratings and reviews, the system processes and stores rating data."

The following is an example of data extraction from the Sequence Diagram used, as follows:

- SD1 = "Select Menu Section, display section, fill in form add section, input message is invalid, display list section, select menu section, display page section, input is invalid, Invalid input message, display section list, display section list, fill form add section, data section, display list section, display list section, data section, input no valid, Save Section data, display Section list, display Section list, display Section list."
- SD2 = "Select Rating Results, Show rating results page, Select View Comments Button, View rating comments, Select Rating Results, Show rating results page, Select View Comments Button, Display rating comments."
- SD3 = "Select Menu Approval Ticket, ticket approval page appears, Press button approve, order ticket does not approve, Message Status Successfully Approves, Press button approve, Select Menu Approval Ticket, approval page appears ticket, confirm not, ticket message not approve, Ticket Status Successfully approve, Message Status Successfully approve, Press button approve, Approve Ticket Status, Ticket Status approve, Ticket Status Successfully approve, Approve Ticket Status, Approve Ticket, confirm not, Ticket Status approve."
- SD4 = "Select Menu Ticket, display ticket page, invalid input message, Ticket Order Saved, Select Menu Ticket, ticket page display, invalid input, invalid input message, Ticket Data Saved, Ticket Order Saved, Saved Ticket Data."
- SD5 = "Select Ticket History Menu, ticket history page appears, Select an item ticket, Show rating form, Fill in Rating Form, Rating Message Saved successfully, Select Ticket History Menu, page appears ticket history, Rating Data Saved Successfully, Rating Message Saved Successfully, Select one ticket item, Show rating form, Fill Rating Form, Ticket Rating Data, Saved Ticket Rating Data, Rating Data Saved Successfully, Ticket Rating Data, Save Rating Data, Saved Ticket Rating Data."

C. Text Pre-Processing

Based on the extraction results from Step Performed and the sequence diagram, the following are the steps for Text Pre-Processing using the Natural Language Toolkit (NLTK) for Python programming, via JupyterLab, which is part of Anaconda Navigator, namely:

- Case Folding. It converts the extracted data to lowercase to simplify data processing. The document used is a

document with a file.txt format whose contents are taken from the Use Case Description and Sequence Diagram. In this process, all the extraction results will be stored in a file named “text_before.txt,” which will enter the Case Folding process and produce a “text_sehasil.txt” file, as shown in Figure 3.

```
# Read files with txt format
elicitation = open('Text_sebelum.txt', 'r')

sentences = [line.lower() for line in elicitation]
with open('Text_sesudah.txt', 'w') as out:
    out.writelines(sorted(sentences))
print(sentences)
```

Fig. 3 Case Folding

- Tokenization. The results of the Case Folding process then go through a tokenization process that aims to break sentences into words [14]. There are five documents for use case descriptions and five documents for sequence diagrams, as shown in Figure 4.

```
# Case Folding
# Mengelompokkan kalimat sesuai dengan masing-masing fitur yang ada pada Use Case Description
sp01 = "Admin selects the Manage Section menu, the system displays the section list, the admin"
sp02 = "Supervisor selects the Rating Results menu, the System displays the rating results page"
sp03 = "The APT officer selects the Approval Ticket menu, the system displays a list of entry t"
sp04 = "The Service User selects the Ticket menu, the system displays the ticket list page, the"
sp05 = "The service user selects the Ticket History menu, the system displays a completed ticke"

# Mengelompokkan kalimat sesuai dengan masing-masing fitur yang ada pada Sequence Diagram
sd1 = "Select Menu Section, display section section, fill in form add section, input message is"
sd2 = "Select Rating Results, Show rating results page, Select View Comments Button, View ratin"
sd3 = "Select Menu Approval Ticket, ticket approval page appears, Press button approve, order ti"
sd4 = "Select Menu Ticket, display ticket page, invalid input message, Ticket Order Saved, Sele"
sd5 = "Select Ticket History Menu, ticket history page appears, Select an item ticket, Show rat"
documents = [sp01, sp02, sp03, sp04, sp05, sd1, sd2, sd3, sd4, sd5]
```

Fig. 4 Tokenization

- Stemming/Lemmatization. This process uses NTLK “wordnet” as the semantic English dictionary. There are ten documents obtained from the tokenization results, which will be normalized and given an index for each word in the document, which can see in Figure 5.

```
# Stemming
import nltk, string, numpy
nltk.download('punkt')
stemmer = nltk.stem.porter.PorterStemmer()
def StemTokens(tokens):
    return [stemmer.stem(token) for token in tokens]
remove_punct_dict = dict((ord(punct), None) for punct in string.punctuation)
def LemNormalize(text):
    return StemTokens(nltk.word_tokenize(text.lower().translate(remove_punct_dict)))

# Lemmatization
nltk.download('wordnet')
lemmer = nltk.stem.WordNetLemmatizer()
def LemTokens(tokens):
    return [lemmer.lemmatize(token) for token in tokens]
remove_punct_dict = dict((ord(punct), None) for punct in string.punctuation)
def LemNormalize(text):
    return LemTokens(nltk.word_tokenize(text.lower().translate(remove_punct_dict)))

# Lemmatization using wordnet with english Dictionary
from sklearn.feature_extraction.text import CountVectorizer
LemVectorizer = CountVectorizer(tokenizer=LemNormalize, stop_words='english')
LemVectorizer.fit_transform(documents)
```

Fig. 5 Stemming / Lemmatization

- Stop Words. It eliminates unnecessary English words in every sentence used [7].

D. Similarity Search

In looking for similarities, several steps must be taken, namely:

- TF-IDF Matrix. TF-IDF functions to calculate the similarity of sentences in each document using Cosine Similarity. If the number in the matrix is close to number

1, it means that the sentence has high similarity. Please look at Figure 6.

```
# Calculating the similarity, which is close to 1 of the two artifac
cos_similarity_matrix = (tfidf_matrix * tfidf_matrix.T).toarray()
print(cos_similarity_matrix)
```

```
[[1. 0.08635251 0.12175137 0.15016071 0.09019879 0.69108792
 0.01562379 0.00713383 0.09311384 0.11698167]
 [0.08635251 1. 0.17573911 0.19543897 0.33290573 0.05752114
 0.68016075 0.0758352 0.1205135 0.36330972]
 [0.12175137 0.17573911 1. 0.338287 0.24799982 0.06556061
 0.0251507 0.41116234 0.22568805 0.13367736]
 [0.15016071 0.19543897 0.338287 1. 0.77700537 0.1089237
 0.03429119 0.24545565 0.36624718 0.28392299]
 [0.09019879 0.33290573 0.24799982 0.77700537 1. 0.07869849
 0.21668268 0.15660251 0.26075339 0.42994545]
```

Fig. 6 Similarity Search

- Similarity Analysis. Based on the results of Cosine Similarity using the TF-IDF Matrix, the highest similarity results from the two artifacts used, namely 0.69108792, as in SP01 and SD01, which can be seen in Table VI.

TABLE IVI SIMILARITY ANALYSIS

	SD1	SD 2	SD 3	SD 4	SD 5
SP 01	0.6910 8792	0.0156 2379	0.0071 3383	0.0931 1384	0.11698 167
SP 02	0.0575 2114	0.6801 6075	0.0758 352	0.1205 135	0.36330 972
SP 03	0.0655 6061	0.0251 507	0.4111 6234	0.2256 8805	0.13367 736
SP 04	0.1089 237	0.0342 9119	0.2454 5565	0.3662 4718	0.28392 299
SP 05	0.0786 9849	0.2166 8268	0.1566 0251	0.2607 5339	0.42994 545

- Stop Word Removal. Used to remove or delete words in SP01 and SD01 documents that are not needed for the text mining process. The result of the SP01 document is “admin selects manage section menu, the system displays section list, admin selects add button, system displays added section form, admin inputs added section form, system stores section data displays table, admin chooses change section item, the system displays section data edit form , admin inputs data changes, the system saves data changes, admin selects delete section items, the system deletes section items.”. Then for SD01 it is "select menu section, display section section, fill form add section, input message invalid, display list section section, select menu section, display page section section, input invalid, invalid input message, display section list, display section list, fill form add section, data section, display list section, display list section, data section, valid input, save section data, display section list, display section list, display section list.”. Please look at Figure 7.

```

# Stopword removal
import io
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize
#nltk.download('stopwords')
stop_words = set(stopwords.words('english'))
file1 = open("Text_sesudah.txt")
line = file1.read()
words = line.split()

for r in words:
    if not r in stop_words:
        appendFile = open('stopword_sesudah.txt', 'a')
        appendFile.write(" "+r)
        appendFile.close()

fo = open("stopword_sesudah.txt", "r+")
string = fo.read()
print("STOP WORD REMOVAL :", string)
fo.close()

```

Fig. 7 Stop Word Removal

- Semantic Relationships Between Words. To calculate the semantics between in "SP01" and "SD01" documents, the Wu-Palmer similarity concept is used through the "WordNet Similarity for Java" application. The following are examples of some of the results obtained from Wu-Palmer, which can be seen in Figure 8.

the	system	displays	the	section	list	the	admin	selects	the	Add	is
DT	NN	VBZ	DT	NN	NN	DT	NN	VBZ	DT	VB	IN
-	-	-	-	-	wup(list#1, menu#3) = 0.9412	-	-	-	-	-	0.3333
-	0.8000	-	-	0.7368	0.9412	-	-	-1	-	-	0
-	0.7368	-	-	1.0000	0.5333	-	-	-1	-	-	0
-	-	-	-	-	-	-	-	-	-	-	0
-	0.7778	-	-	0.6667	0.6154	-	-	-1	-	-	0
-	0.7368	-	-	1.0000	0.5333	-	-	-1	-	-	0
-	0.7368	-	-	1.0000	0.5333	-	-	-1	-	-	0

Fig. 8 Semantic Relationship Between Words

E. Validity and Reliability Testing

There are two values obtained from Gwet's AC1 formula, namely from the Python program code and from a questionnaire filled out by experts. The value obtained from the program code is 0.12362, which means "Slight Agreement." Please look at Figure 9.

```

# Kappa Score
from sklearn.metrics import cohen_kappa_score
cohen_kappa_score(tf_matrix[4], tf_matrix[8])

0.12362637362637374

```

Fig. 9 Validity and Reliability Testing

Meanwhile, the results for filling out the questionnaire by experts who already have experience and understand UML in the SRS document can be seen in Table VII.

TABLE V SIMILARITY ANALYSIS

Expert 1	Expert 2		
	Yes	No	Total
Yes	43	0	43
No	2	6	8
Total	45	6	50

The first step is to calculate the results of filling out the questionnaire to calculate the observed agreement (P) between the two experts based on Table VI.

$$P = \frac{43 + 6}{50} = 0.98$$

The next step is to calculate the Chance-Agreement or $e(Y)$.

$$P_1 = \frac{(45 + 43)/2}{50} = 0.88$$

$$e(Y) = 2(0.88)(1 - 0.88) = 0.2112$$

After getting the results from the calculation of the observed agreement (P) and the calculation of Chance-Agreement or $e(Y)$. Next is to calculate AC1 statistics using Aggregation Coefficients.

$$AC1 = \frac{0.98 - 0.2112}{1 - 0.2112} = 0.97464$$

The result of Gwet's AC1 formula through a questionnaire filled in by the expert is 0.97464, which means "Almost perfect." Based on these results, there is a difference between the Kappa Score value from the Python program code, which is 0.12362, and from the questionnaire filled in by the expert, which is 0.97464.

V. CONCLUSIONS AND FUTURE WORK

This research was conducted to extract in determine the suitability between the Step Performed and Sequence Diagram on the SRS document for the APTU KPKNL Bandung application. The results of this extraction process implementation are divided into several documents contained in the Use Case Description based on Step Performed, namely: SP01, SP02, SP03, SP04, and SP 05. As for the Sequence Diagram, namely: SD1, SD2, SD3, SD4, and SD5. Three things can be concluded as the core results of this research activity they are as follows:

- The highest similarity between documents is in the SP01 and SD01 documents, with the similarity value being 0.69108792.
- The highest similarity between words is found in the words "list" and "menu," with a similarity value of 0.9412.
- Kappa Score from Gwet's AC1 formula using the Python programming language is 0.12362, which means "Slight Agreement," while the Kappa Score value using a questionnaire filled in by the Expert is 0.97464, which means "Almost perfect."

For future work, the results of this study can be used as a reference for development in further activities, namely:

- The text mining process can be elaborated using deep learning.
- The calculating similarity between two different data can be elaborated using Containment Measure, Overlap Coefficient, and Jaccard Similarity.

Calculating the Validity and Reliability of Gwet's AC1 formula, which can also be combined with several algorithms such as BP Coffee and Crippen Alpha.

ACKNOWLEDGMENT

This work was supported by the Department of Informatics and Department of Software Engineering - School of Computing at Telkom University, Bandung 40257.

REFERENCES

- [1] A. Azzam, Y. Priyadi and J. H. Husen, "Similarity Software Requirement Specification (SRS) Elicitation Based on the Requirement Statement Using Text Mining

- on the MNC Play Inventory Management Application," 2021 4th International Conference of Computer and Informatics Engineering (IC2IE), 2021, pp. 123-128, doi: 10.1109/IC2IE53219.2021.9649023.
- [2] Y. Priyadi, A. M. Putra and P. S. Lyanda, "The similarity of Elicitation Software Requirements Specification in Student Learning Applications of SMKN7 Baleendah Based on Use Case Diagrams Using Text Mining," 2021 IEEE 5th International Conference on Information Technology, Information Systems and Electrical Engineering (ICITISEE), 2021, pp. 115-120, doi: 10.1109/ICITISEE53823.2021.9655844.
 - [3] R. Mohanani, P. Ralph, B. Turhan, and V. Mandic, "How Templated Requirements Specifications Inhibit Creativity in Software Engineering," IEEE Trans. Softw. Eng., pp. 1-1, 2021, doi: 10.1109/TSE.2021.3112503.
 - [4] F. Hujainah, R. B. A. Bakar, M. A. Abdulgaber, and K. Z. Zamli, "Software Requirements Prioritisation: A Systematic Literature Review on Significance, Stakeholders, Techniques and Challenges," IEEE Access, vol. 6, pp. 71497-71523, 2018, doi: 10.1109/ACCESS.2018.2881755.
 - [5] M. Asif, I. Ali, M. S. A. Malik, M. H. Chaudary, S. Tayyaba, and M. T. Mahmood, "Annotation of Software Requirements Specification (SRS), Extractions of Nonfunctional Requirements, and Measurement of Their Tradeoff," IEEE Access, vol. 7, pp. 36164-36176, 2019, doi: 10.1109/ACCESS.2019.2903133.
 - [6] M. Sudhamani and L. Rangarajan, "Code similarity detection through control statement and program features," Expert Syst. Appl., vol. 132, pp. 63-75, Oct. 2019, doi: 10.1016/j.eswa.2019.04.045.
 - [7] Y. Cai, Q. Zhang, W. Lu, and X. Che, "A hybrid approach for measuring semantic similarity based on IC-weighted path distance in WordNet," J. Intell. Inf. Syst., vol. 51, no. 1, pp. 23-47, Aug. 2018, doi: 10.1007/s10844-017-0479-y.
 - [8] M. Younas, D. N. A. Jawawi, I. Ghani, and M. A. Shah, "Extraction of non-functional requirement using semantic similarity distance," Neural Comput. Appl., vol. 32, no. 11, pp. 7383-7397, Jun. 2020, doi: 10.1007/s00521-019-04226-5.
 - [9] R.P. Octavially, Y. Priyadi, and S. Widowati, "Extraction of Activity Diagrams Based on Steps Performed in Use Case Description Using Text Mining (Case Study: SRS Myoffice Application)," International Conference on Electrical and Electronic Intelegant System (ICE3IS), 2022.
 - [10] E. J. Sari, Y. Priyadi, and R. R. Riskiana, "Implementation of Semantic Textual Similarity Between Requirement Specification and Use Case Description Using WUP Method (Case Study: Sipjabs Application)," 2022 IEEE World AI IoT Congress (AIIoT), 2022.
 - [11] C. Dreisbach, T. A. Koleck, P. E. Bourne, and S. Bakken, "A systematic review of natural language processing and text mining of symptoms from electronic patient-authored text data," Int. J. Med. Inf., vol. 125, pp. 37-46, May 2019, doi: 10.1016/j.ijmedinf.2019.02.008.
 - [12] NMAM Institute of Technology, Dept. of Computer Science and Engineering, A. S. Nayak, and A. P. Kanive, "Survey on Pre-Processing Techniques for Text Mining," Int. J. Eng. Comput. Sci., Jun. 2016, doi: 10.18535/ijecs/v5i6.25.
 - [13] G. Orellana, B. Arias, M. Orellana, V. Saquicela, F. Baculima, and N. Piedra, "A study on the impact of pre-processing techniques in Spanish and english text classification over short and large text documents," Proc. - 3rd Int. Conf. Inf. Syst. Comput. Sci. INCISCOS 2018, vol. 2018-Decem, pp. 277-283, 2018.
 - [14] D. Soyusiawaty and Y. Zakaria, "Book data content similarity detector with cosine similarity (case study on digilib.uad.ac.id)," Proceeding 2018 12th Int. Conf. Telecommun. Syst. Serv. Appl. TSSA 2018, pp. 1-6, 2018.
 - [15] J. A. Pamungkas, Y. Priyadi, and M. J. Alibasa, "Measurement of Similarity Between Requirement Elicitation and Requirement Specification Using Text Pre-Processing in the Cinemaloka Application," 2022 IEEE World AI IoT Congress (AIIoT), 2022.
 - [16] T. A. Hutajulu, Y. Priyadi and A. Gandhi, "Text Data Processing in Requirement Specifications as a Reference for Similarities Between Use Case Diagrams and Use Case Descriptions for Smart Sleeping Lamp Application Documents," 2022 IEEE World AI IoT Congress (AIIoT), 2022.
 - [17] M. Asif, I. Ali, M. S. A. Malik, M. H. Chaudary, S. Tayyaba, and M. T. Mahmood, "Annotation of Software Requirements Specification (SRS), Extractions of Nonfunctional Requirements, and Measurement of Their Tradeoff," IEEE Access, vol. 7, pp. 36164-36176, 2019, doi: 10.1109/ACCESS.2019.2903133.
 - [18] R. Z. I. Yanis, Y. Priyadi, and S. Y. Puspitasari, "Measurement of Similarity between Use Case Description and Sequence Diagram in Software Requirement Specification using Text Analysis for Dtrain Application," International Conference on Electrical and Electronic Intelegant System (ICE3IS), 2022.
 - [19] M. Younas, D. N. A. Jawawi, I. Ghani, and M. A. Shah, "Extraction of non-functional requirement using semantic similarity distance," Neural Comput. Appl., vol. 32, no. 11, pp. 7383-7397, Jun. 2020, doi: 10.1007/s00521-019-04226-5.
 - [20] A. Karnik, S. Goswami, and R. Guha, "Detecting Obfuscated Viruses Using Cosine Similarity Analysis," in First Asia International Conference on Modelling & Simulation (AMS'07), Phuket, Mar. 2007, pp. 165-170. doi: 10.1109/AMS.2007.31.
 - [21] Wei, G. (2017). Some Cosine Similarity Measures for Picture Fuzzy Sets and Their Applications to Strategic Decision Making. Informatica (Netherlands), 28(3), 547-564. <https://doi.org/10.15388/Informatica.2017.144>.
 - [22] G. Wei, "Some Cosine Similarity Measures for Picture Fuzzy Sets and Their Applications to Strategic Decision Making," Informatica, vol. 28, no. 3, pp. 547-564, Jan. 2017, doi: 10.15388/Informatica.2017.144.
 - [23] E. Haddi, X. Liu, and Y. Shi, "The role of text pre-processing in sentiment analysis," Procedia Comput. Sci., vol. 17, pp. 26-32, 2013